

Multitask programming

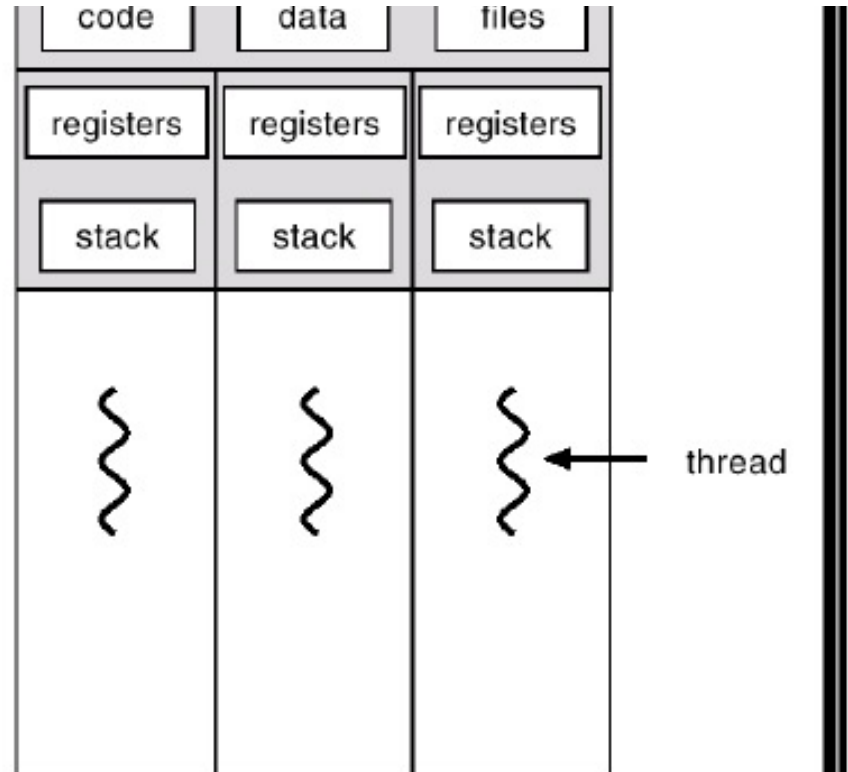
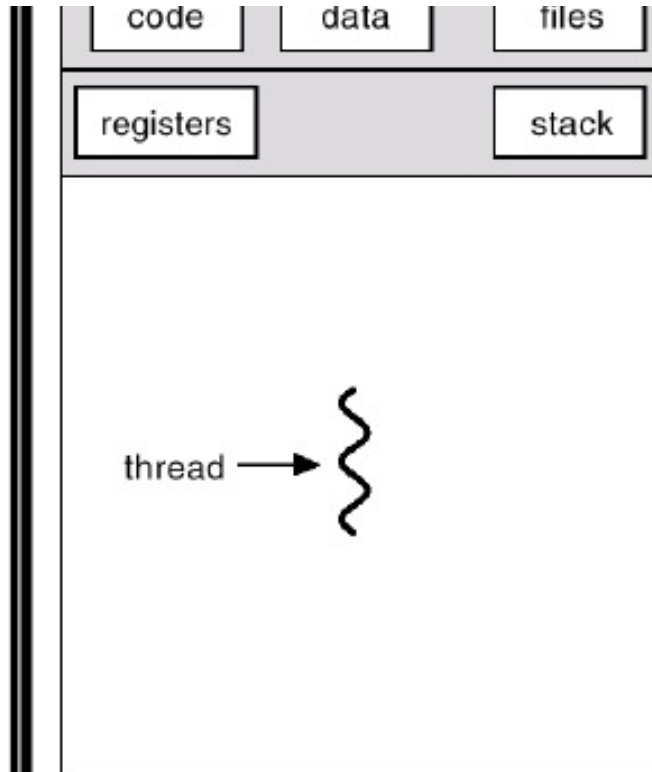
Processo

- Ambiente di esecuzione *self-contained* con un proprio insieme di risorse
 - Memoria
 - Dati: Stack e heap
 - Codice
- I processi comunicano tra di loro per realizzare compiti complessi (inter-process communication, socket)

Thread

- Detto anche processo leggero, è una unità di esecuzione che consiste
 - Stack
 - Un insieme di registri
- Condivide con altri thread la sezione codice, la sezione dati (heap) e le risorse che servono per la loro esecuzione
- Rendono più efficiente l'esecuzione di attività che condividono lo stesso codice
 - Context-switch è più veloce rispetto a quello dei processi
- I thread non sono tra loro indipendenti perché condividono codice e dati
- E' necessario che le operazioni non generino conflitti tra i diversi thread di un task

Thread



- Tutti i programmi Java comprendono almeno un thread
 - Un programma costituito solo dal metodo main viene eseguito come un singolo thread
- Java fornisce
 - Classi e interfacce che consentono di creare e manipolare thread aggiuntivi nel programma
 - Meccanismi nel linguaggio per la sincronizzazione delle risorse

- Esistono due modi per implementare in Java un thread
 - Definire una classe estendendola dalla classe `java.lang.Thread`
 - Definire una classe implementando l'interfaccia `java.lang.Runnable`

Creazione di threads: stile 1

- definiamo una sottoclasse di Thread che ridefinisce il metodo run()
 - il metodo contiene il codice che vogliamo eseguire nel thread
 - la sottoclasse non ha bisogno di avere un costruttore
 - all'atto della creazione di un nuovo oggetto si chiama automaticamente il costruttore Thread()
 - dopo aver creato l'oggetto della sottoclasse, il codice parte invocando il metodo start()

Un esempio di thread stupido 1

- cosa fa il metodo `run()` che contiene il codice che vogliamo eseguire nel thread
 - visualizza il thread corrente
 - stampa nell'ordine i numeri da 0 a 4, con un intervallo di 1 secondo
 - l'attesa viene realizzata con il metodo statico `sleep()` che deve essere incluso in un `try` perché può sollevare l'eccezione `InterruptedException` se interrotto da un altro thread
 - visualizza il messaggio "Fine run"

Creazione di threads stile 1: esempio il thread

- la sottoclasse di Thread

```
public class MioThread extends Thread {  
    public void run(){  
        System.out.println ("Thread run" + Thread.currentThread ( ));  
        for (int i = 0; i < 5; i++) {  
            System.out.println (i);  
            try {  
                Thread.currentThread ( ).sleep (1000);  
            } catch (InterruptedException e) {    } }  
        System.out.println ("Fine run");  
    }  
}
```

Creazione di threads stile 1: esempio il programma “principale”

1. visualizza il thread corrente
2. crea e manda in esecuzione il thread
3. fa dormire il thread corrente per 2 secondi
4. visualizza il messaggio “Fine main”
5. termina

```
public class ProvaThread {  
    public static void main (String argv[ ]) {  
        System.out.println ("Thread corrente: " + Thread.currentThread ( ));  
        MioThread t = new MioThread ( );  
        t.start ( );  
        try {  
            Thread.sleep (2000);  
        } catch (InterruptedException e) { }  
        System.out.println ("Fine main"); }  
}
```

Creazione di threads stile 1: esempio il risultato

Thread corrente: Thread[main,5,system]

Thread run: Thread[Thread-0,5,system]

0

1

Fine main

2

3

4

Fine run

Creazione di threads: stile 2

- definiamo una classe `c` che implementa l'interfaccia `Runnable`
 - nella classe deve essere definito il metodo `run()`
 - non è necessario che siano definiti costruttori
- dopo aver creato un oggetto di tipo `c`, creiamo un nuovo oggetto di tipo `Thread` passando come argomento al costruttore `Thread(Runnable t)` l'oggetto di tipo `c`
- il thread (codice del metodo `run()` di `c`) viene attivato eseguendo il metodo `start()` dell'oggetto di tipo `Thread`

Creazione di threads stile 2: esempio

```
public class Prova implements Runnable {  
    public void run(){  
        System.out.println ("Thread run" + Thread.currentThread ( ));  
        for (int i = 0; i < 5; i++) {  
            System.out.println (i);  
            try {Thread.currentThread ( ).sleep (1000); }  
            catch (InterruptedException e) {    } }  
        System.out.println ("Fine run");}  
    }  
}
```

Creazione di threads stile 2: esempio

```
public class ProvaThread {  
    public static void main (String argv[ ]) {  
        System.out.println ("Thread corrente: " + Thread.currentThread ( ));  
        Prova pt = new Prova ( );  
        Thread t = new Thread(pt);  
        t.start ( );  
        try {  
            Thread.sleep (2000);  
        } catch (InterruptedException e) { }  
        System.out.println ("Fine main"); }  
}
```

CONTROLLO DEI THREAD

- Un thread continua la propria esecuzione:
 - fino alla sua terminazione naturale
 - fino a che non viene *espressamente fermato* da un altro thread che invochi su di esso il metodo `stop()`
- un thread può anche essere *sospeso* temporaneamente dalla sua esecuzione
 - perché un'operazione blocca l'esecuzione (es. una operazione di input che implica di attendere dei dati). Il Thread si blocca finchè i dati non sono disponibili.
 - perché invoca la funzione `Thread.sleep()`;
 - da un altro thread che invochi su di esso il metodo `suspend()`, per *riprendere* poi la propria esecuzione quando qualcuno invoca su di esso il metodo `resume()`.

stop, suspend, resume: PROBLEMI

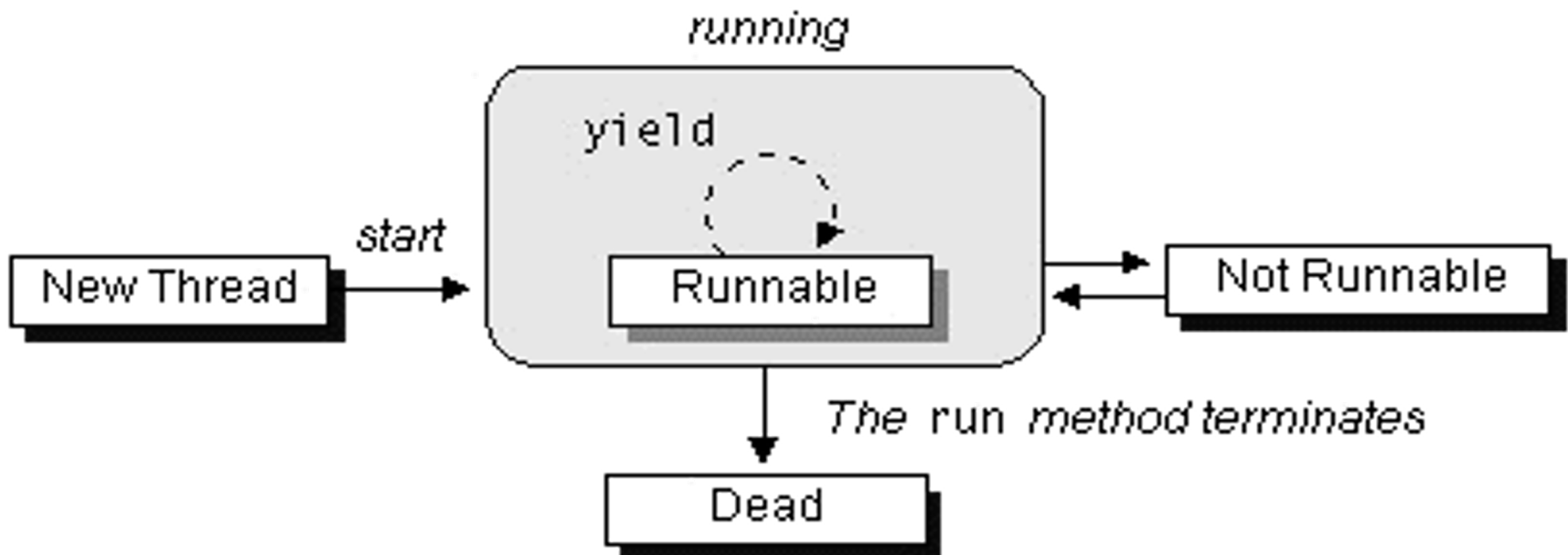
- **ATTENZIONE!** stop(), suspend() e resume() sono deprecati in Java2, in quanto *non sicuri*.
- Questi tre metodi *agiscono in modo brutale* su un thread (che non può "difendersi" in nessun modo), e così facendo **rischiano di creare situazioni di blocco (deadlock)**:
 - se il thread sospeso / fermato stava facendo un'operazione critica, essa *rimane a metà* e l'applicazione viene a trovarsi in uno stato *inconsistente*
 - se il thread sospeso / fermato aveva acquisito una risorsa, essa *rimane bloccata da quel thread* e nessun altro può conquistarla
 - se a questo punto un altro thread si pone in attesa di tale risorsa, si crea una situazione di deadlock.

stop, suspend, resume: PROBLEMI

- LA SOLUZIONE
- La soluzione suggerita da Java 2 consiste nell'adottare un *diverso approccio*:
 - *non* agire d'imperio su un thread,
 - ma *segnalargli l'opportunità* di sospendersi / terminare,
 - **lasciando però al thread il compito di *sospendersi / terminare*,**
 - in modo che possa farlo in un momento "non critico".
 - la funzione *yield()* permette a un thread di cedere l'esecuzione ad un altro!

Vita di un thread

- Il seguente diagramma mostra gli stati in cui si può trovare un thread Java durante la sua vita.
- Illustra anche quali metodi possono causare una transizione da uno stato ad un'altro.



Vita di un thread

New Thread

- Thread in tale stato è un oggetto vuoto ovvero nessuna risorsa di sistema è stata ancora allocata
 - Unico metodo invocabile è start
 - L'invocazione di qualsiasi altro metodo causa l'eccezione `IllegalThreadStateException`

Runnable

- Metodo start crea le risorse di sistema necessarie per eseguire il thread e invoca il metodo `run()`

Not Runnable:

E' in tale stato a seguito di uno dei seguenti eventi

- Metodo `sleep` invocato
- Viene invocato il metodo `wait` al verificarsi di una determinata condizione
- Thread in blocco su una operazione di I/O

Dead

- E' il termine dell'esecuzione del metodo `run`
- All'interno della classe `Thread` esiste il metodo `stop` **NON UTILIZZARE**

Testing stato dei Thread

- In Java 5 è stato introdotto il metodo `getState` all'interno della classe `Thread`. Il valore ritornato è:
 - `NEW`
 - `RUNNABLE`
 - `BLOCKED`
 - `WAITING`
 - `TIMED_WAITING`
 - `TERMINATED`
- Inoltre, esiste (prima della versione 5) il metodo `isAlive`
 - Ritorna `true` se il thread è started ma non è stopped (stato `Runnable` o `Not Runnable`)
 - Ritorna `false` se il thread è negli stati `New Thread` o `Dead`
 - Non è possibile differenziare tra i vari stati

Thread Scheduling

- Il sistema di run-time di Java supporta un semplice algoritmo di scheduling chiamato *a priorità-fissa*
- L'algoritmo schedula i thread con lo stato Runnable sulla base della loro priorità
- Quando un thread viene creato eredita la priorità del thread che lo sta creando. E' possibile modificare la priorità utilizzando il metodo `setPriority` (costanti `MIN_PRIORITY` e `MAX_PRIORITY` definite all'interno della classe `Thread`)
- In un dato istante di tempo quando deve essere determinato il thread da eseguire, il sistema di run-time sceglie il thread con stato Runnable con priorità più alta
- Soltanto quando il thread finisce, oppure viene invocato il metodo `yield` oppure diventa Not Runnable un thread con priorità più bassa viene eseguito.
- Se vi sono thread con la stessa priorità lo scheduler ne sceglie arbitrariamente uno

Thread Scheduling

- Il thread scelto viene eseguito finché una delle seguenti condizioni si verificano
 - Un thread con priorità più alta diventa Runnable
 - Viene invocato il metodo yield oppure termina il metodo run
 - Nei sistemi che supportano il time-slicing, lo slot di tempo del thread è terminato
- Lo scheduler è anche *preemptive* ovvero che se in un determinato istante di tempo un thread con priorità più alta diventa Runnable allora il sistema di runtime sceglie quello con priorità più alta (il nuovo thread si dice che preempt gli altri thread)
- In un determinato istante di tempo il thread con priorità più alta viene eseguito. Tuttavia questo non è garantito ovvero lo scheduler potrebbe scegliere un thread con priorità più bassa per eliminare starvation. Utilizzare la priorità soltanto per ragioni di eventuale efficienza e non di correttezza.

■ `yelds()`

- Serve per dare un *suggerimento allo scheduler in modo che possa assegnare la CPU ad un altro thread*

■ `sleep()` e `interrupt()`

- Si può mettere un thread in uno stato d'attesa con il metodo statico `sleep` che prende il numero di millisecondi da aspettare
- Un oggetto può ottenere il suo thread d'esecuzione usando il metodo `Thread.currentThread()`
- Se un thread è in stato d'attesa, lo si può svegliare chiamando il metodo **`interrupt()`**
- Tale metodo genera un'eccezione **`InterruptedException`**. Per questo è necessario ogni volta che si fa uno `sleep` prevedere l'eccezione **`InterruptedException`**

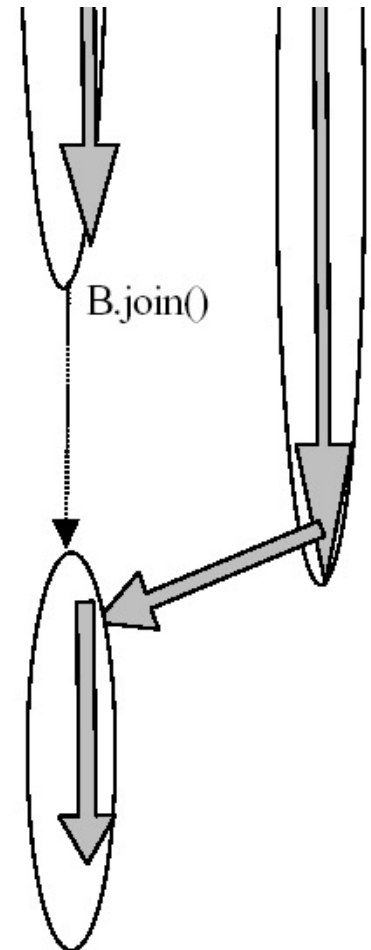
CONTROLLO DEI THREAD: JOIN

Si può decidere di aspettare che termini l'esecuzione di un thread.

Per fare questo, bisogna invocare l'operazione `join()` sul thread che si intende aspettare

Quando però A fa `B.join()`, la sua esecuzione si sospende nell'attesa che il Thread B termini la sua esecuzione

E' come se ci fosse un ri-congiungimento dei flussi di esecuzione



Esempio

```
class Esempio4 {  
    public static void main(String args[]){  
        System.out.println("Main - inizio");  
        while(true) {  
            MyThread t = new  
            System.out.print("n  
            System.out.println(''  
            t.start();  
            try {  
                t.join(); /  
            }catch(Interrupte  
        }  
    }  
}
```

- il main è un ciclo infinito che crea continuamente nuovi thread (fino a che non viene chiuso con CTRL-C)
- prima di creare un nuovo thread, il main attende, tramite join(), che termini quello precedentemente in esecuzione
- l'attesa di join() potrebbe essere interrotta da un'eccezione
- necessità di try/catch

```
class MyThread extends Thread {  
    public void run(){  
        System.out.println("Qui thread " +  
        Thread.currentThread().getName() );  
    }  
}
```

Come rendere un programma multithreaded

- Riassumendo: ci sono 2 modi per rendere un programma multithreaded...
 - Una classe implements **Runnable** e definisce il metodo **run()**. Questa classe viene passata nel costruttore di un **Thread** e viene chiamato il metodo **start()** per eseguire il metodo **run()**.
 - Estendere la classe **Thread** ed eseguire l'overriding del metodo **run()**.

Thread User e Thread Daemon

- I thread deamon sono thread
 - Che effettuano operazioni ad uso di altri thread
 - Per esempio: Garbage collector
 - Sono eseguiti in background
 - Usano cioè il processore solo quando altrimenti il tempo macchina andrebbe perso
 - A differenza degli altri thread non impediscono la terminazione di una applicazione
 - Quando sono attivi solo thread daemon il programma termina
 - Occorre specificare che un thread è un daemon prima dell'invocazione di start
 - `setDaemon( true );`
 - `isDaemon()`
 - Restituisce true se il thread è un daemon thread

- Un *daemon* thread serve *user threads*.
- Un'applicazione continua ad essere eseguita fino a quando almeno uno dei suoi user thread è vivo.
- Quando un user thread termina in una applicazione, la JVM cerca se ci sono altri user threads ancora vivi.
 - Se si, l'applicazione continua, altrimenti l'applicazione termina perchè la JVM stessa termina.

User e Daemon threads

■ Il seguente codice illustra daemon e user threads

```
class NotRunForever implements Runnable
{
    // il thread main svolge il ruolo di user o non di daemon
    public static void main (String [] args) {
        NotRunForever me = new NotRunForever();
        Thread t1 = new Thread(me);
        t1.setDaemon(true); // t1 è il daemon
        t1.start();
        try {Thread.sleep(50);        }
        catch (InterruptedException e) {System.err.println(e);}
        System.out.println("il thread main termina");
    }
    public void run() {
        while (true) {
            System.out.println("il thread 2 è in esecuzione");
            try {Thread.sleep(10);}
            catch (InterruptedException e) {System.err.println(e);}
        }
    }
}
```

User and Daemon threads

- Nell'esempio precedente, il thread main è il solo user thread dell'applicazione, così l'applicazione termina quando il main termina
- Il metodo `setDaemon` deve essere invocato prima che il thread sia started
- Il JVM's garbage collector viene eseguito come un daemon thread.

Thread groups

- ❑ Ogni thread Java è un membro di un *thread group*.
- ❑ Il Thread groups fornisce un meccanismo per collezionare multiple threads in un singolo oggetto e manipolare questi threads tutti insieme piuttosto che individualmente.
- ❑ Per esempio, è possibile eseguire una chiamata al metodo start o suspend di tutti i threads all'interno di un group con la chiamata di un singolo metodo
- ❑ Java thread groups sono implementati dalla classe **java.lang.ThreadGroup**.

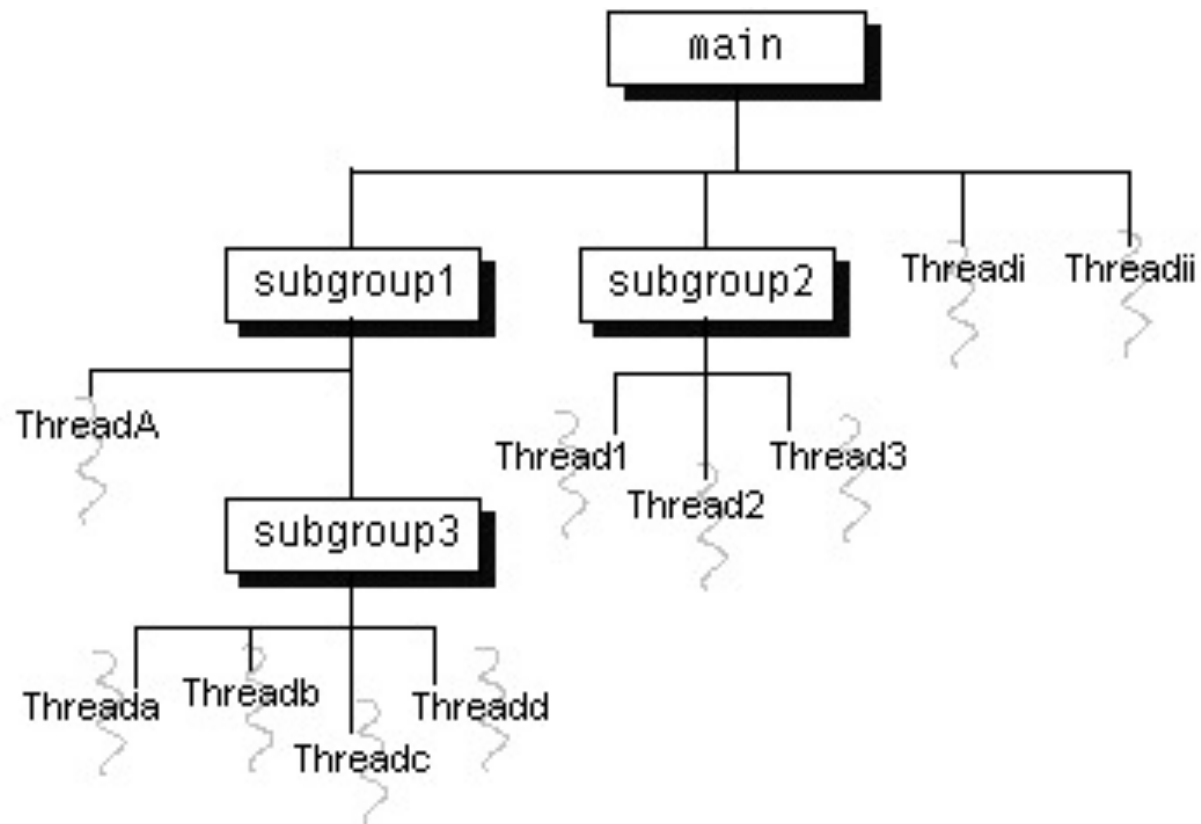
Thread groups

- Il runtime system pone un thread in un thread group durante la costruzione di un thread.
- Quando viene creato un thread, si può:
 - Permettere al runtime system di porre nuovi thread in un default group
 - O si può esplicitamente selezionare nuovi thread's group.
- Il thread è un membro permanente del gruppo in cui è inserito all'atto della creazione
 - Non è possibile spostare un thread in un nuovo gruppo dopo la sua creazione.

Default Thread Group

- Se si crea un nuovo Thread senza specificare il suo gruppo nel costruttore, il runtime system automaticamente pone il nuovo thread nello stesso gruppo (*current thread group*) del thread che lo ha creato (*current thread*)
- Quando un'applicazione Java inizia, il Java runtime system crea un **ThreadGroup** di nome **main**.
- Se non specificato altrimenti, tutti i nuovi threads creati diventano membri del **main** thread group.
 - NOTA: se si crea un thread all'interno di un applet, il nuovo thread's group può essere diverso dal **main**, a seconda del browser o viewer che esegue l'applet.

Default Thread Group



Specifica di un thread group

- Per inserire nuovi thread in un thread group bisogna specificarlo esplicitamente quando si crea il thread.
- La classe **Thread** ha tre costruttori che permettono di settare un nuovo gruppo di thread:

public Thread(ThreadGroup *group*, Runnable *runnable*)

public Thread(ThreadGroup *group*, String *name*)

public Thread(ThreadGroup *group*, Runnable *runnable*, String *name*)

- crea un nuovo thread
- lo inizializza utilizzando i parametri **Runnable** e **String**
- inserisce il nuovo thread in uno specifico gruppo

Specifica di un thread group

- Il seguente codice crea un threadgroup (**myThreadGroup** e inserisce un thread (**myThread**) nel gruppo:

```
ThreadGroup myThreadGroup = new ThreadGroup("My Group of Threads");
```

```
Thread myThread = new Thread(myThreadGroup, "a thread for my group");
```

- Il **ThreadGroup** passato al costruttore di un **Thread** non deve essere necessariamente creato dal programmatore ma è possibile utilizzare gruppi creati da Java runtime system o gruppi creati dall'applicazione nella quale l'applet è in esecuzione.

Quale è il threadgroup di un thread?

- Per conoscere il threadgroup di un thread esiste il metodo **getThreadGroup** method:

```
theGroup = myThread.getThreadGroup();
```

- Conosciuto il **ThreadGroup**, si possono reperire informazioni sul threadgroup

Sincronizzazione di Thread

- Threads asincroni
 - Ogni thread contiene tutti i dati e i metodi necessari per l'esecuzione
 - I threads evolvono indipendentemente
 - Nessun problema per le velocità relative dei threads.
- Cosa accade quando esistono dati condivisi tra più threads?
 - Problema produttore/consumatore

Sincronizzazione

- la sincronizzazione permette di evitare l'esecuzione contemporanea di parti di codice delicate (Problemi di Competizione)
 - Realizzazione di sezioni critiche
- il multithreading può essere sfruttato per risolvere anche Problemi di Cooperazione (esecuzione di attività comune mediante scambio di informazioni)
 - esempio classico: la relazione produttore-consumatore
 - il thread consumatore deve attendere che i dati da utilizzare vengano prodotti
 - il thread produttore deve essere sicuro che il consumatore sia pronto a ricevere per evitare perdita di dati

Sincronizzazione: Monitors

- Java fornisce come supporto alla sincronizzazione tra thread (competizione e/o cooperazione) il costrutto Monitor
- Dunque vengono forniti
 - Mutex (contenuto in `java.lang.Object`)
 - Condition variables (contenuto in `java.lang.Object`)

Mutual Exclusion

- Java fornisce il meccanismo di sincronizzazione dei mutex (contrazione di mutual exclusion) per l'accesso a sezioni critiche
 - un mutex è una risorsa del sistema che può essere posseduta da un solo thread alla volta
 - ogni istanza di qualsiasi oggetto ha associato un mutex
- quando un thread esegue un metodo che è stato dichiarato sincronizzato mediante il modificatore synchronized
 - entra in possesso del mutex associato all'istanza
 - Il thread che blocca la mutex ("mutex lock") acquisisce l'accesso esclusivo alla sezione critica
 - Eventuali thread che vogliano accedere alla sezione critica saranno posti in uno stato di attesa (blocked)

Sincronizzazione

- Java implementa un meccanismo simile al monitor per garantire la sincronizzazione fra thread
- Ogni oggetto ha un lock associato ad esso
 - Nelle classi possono essere definiti metodi `synchronized`
`synchronized int myMethod()`
 - Un solo thread alla volta può eseguire un metodo `synchronized`
 - Se un oggetto ha più di un metodo sincronizzato, comunque uno solo può essere attivo

- La chiamata di un metodo sincronizzato richiede il “possesso” del lock
- Se un thread chiamante non possiede il lock (un altro thread ne è già in possesso), il thread viene sospeso in attesa del lock
- Il lock è rilasciato quando un thread esce da un metodo sincronizzato

Sincronizzazione: esempio 1

```
public class ProvaThread2 implements Runnable {  
    public static void main (String argv[ ]) {  
        ProvaThread2 pt = new ProvaThread2 ();  
        Thread t = new Thread(pt);  
        t.start ( );  
        pt.m2(); }  
    public void run(){ m1();}  
    synchronized void m1 ( ) {  
        ... }  
    void m2 ( ) {  
        ... } } }
```

due metodi, m1 e m2, vengono invocati contemporaneamente da due threads su uno stesso oggetto pt
m1 è dichiarato synchronized mentre m2 no
il mutex associato a pt viene acquisito all'ingresso del metodo m1 non blocca l'esecuzione di m2 in quanto esso non tenta di acquisire il mutex

Sincronizzazione: esempio 2

```
public class ProvaThread2 implements Runnable {  
    public static void main (String argv[ ]) {  
        ProvaThread2 pt = new ProvaThread2 ( );  
        Thread t = new Thread(pt);  
        t.start ( );  
        pt.m2();  
    }  
  
    public void run(){ m1();}  
    synchronized void m1 ( ) {  
        for (char c = 'A'; c < 'F'; c++) {  
            System.out.println (c);  
            try { Thread.sleep (1000); }  
            catch (InterruptedException e) { }  
        }  
    }  
    void m2 ( ) {  
        for (char c = '1'; c < '6'; c++) {  
            System.out.println (c);  
            try {Thread.sleep (1000); }  
            catch (InterruptedException e) { }  
        }  
    }  
}
```

Sincronizzazione esempio: risultati

1

A

2

B

3

C

4

D

5

E

Sincronizzazione: esempio 3

- se si dichiara `synchronized` anche il metodo `m2`, si hanno due threads che tentano di acquisire lo stesso mutex
- i due metodi vengono eseguiti in sequenza, producendo il risultato

1

2

3

4

5

A

B

C

D

E

Sincronizzazione: esempio 4

- il mutex è associato all'istanza
 - se due threads invocano lo stesso metodo sincronizzato su due istanze diverse, essi vengono eseguiti contemporaneamente

```
public class ProvaThread3 implements Runnable {  
    public static void main (String argv[ ]) {  
        ProvaThread3 pt = new ProvaThread3 ( );  
        ProvaThread3 pt2 = new ProvaThread3 ( );  
        Thread t = new Thread(pt);  
        t.start ( );  
        pt2.m1(); }  
    public void run(){ m1();}  
    synchronized void m1 ( ) {  
        for (char c = 'A'; c < 'F'; c++) {  
            System.out.println (c);  
            try { Thread.sleep (1000); }  
            catch (InterruptedException e) { } } } }
```


Sincronizzazione esempio: risultati

A

A

B

B

C

C

D

D

E

E

Sincronizzazione di metodi statici

- ❑ anche i metodi statici possono essere dichiarati sincronizzati
 - ❑ poiché essi non sono legati ad alcuna istanza, viene acquisito il mutex associato all'istanza della classe `Class` che descrive la classe
- ❑ se invochiamo due metodi statici sincronizzati di una stessa classe da due threads diversi
 - ❑ essi verranno eseguiti in sequenza
- ❑ se invochiamo un metodo statico e un metodo di istanza, entrambi sincronizzati, di una stessa classe
 - ❑ essi verranno eseguiti contemporaneamente

Sincronizzazione con metodi statici: esempio 1

```
public class ProvaThread4 implements Runnable {  
    public static void main (String argv[ ]) {  
        ProvaThread4 pt = new ProvaThread4 ( );  
        Thread t = new Thread(pt);  
        t.start ( );  
        m2(); }  
    public void run(){ m1();}  
    synchronized void m1 ( ) {  
        for (char c = 'A'; c < 'F'; c++) {  
            System.out.println (c);  
            try { Thread.sleep (1000); }  
            catch (InterruptedException e) { } } }  
    static synchronized void m2 ( ) {  
        for (char c = '1'; c < '6'; c++) {  
            System.out.println (c);  
            try { Thread.sleep (1000); }  
            catch (InterruptedException e) { } } }  
}
```

Sincronizzazione con metodi statici: esempio 2

■ il risultato

1

A

2

B

3

C

4

D

5

E

Sincronizzazione implicita

- ▣ se una classe non ha metodi sincronizzati ma si desidera evitare l'accesso contemporaneo a uno o più metodi
 - ▣ è possibile acquisire il mutex di una determinata istanza racchiudendo le invocazioni dei metodi da sincronizzare in un blocco sincronizzato
- ▣ struttura dei blocchi sincronizzati

```
private Object mioLock=new Object();
```

```
synchronized (mioLock) {
```

```
    comando 1;
```

```
    ...
```

```
    comando n;}
```

- ▣ la gestione di programmi multithread è semplificata poiché il programmatore non ha la preoccupazione di rilasciare il mutex ogni volta che un metodo termina normalmente o a causa di una eccezione, in quanto questa operazione viene eseguita automaticamente

Sincronizzazione implicita: esempio

```
public class ProvaThread5 implements Runnable {  
    public static void main (String argv[ ]) {  
        ProvaThread5 pt = new ProvaThread5 ( );  
        Thread t = new Thread(pt);  
        t.start ( );  
        synchronized (pt) { pt.m2(); }  
    }  
    public void run(){ m1(); }
```

```
synchronized void m1 ( ) {  
    for (char c = 'A'; c < 'F'; c++) {  
        System.out.println (c);  
        try { Thread.sleep (1000); }  
        catch (InterruptedException e) { } } }
```

```
void m2 ( ) {  
    for (char c = '1'; c < '6'; c++) {  
        System.out.println (c);  
        try {Thread.sleep (1000); }  
        catch (InterruptedException e) { } } }
```

- sequenzializza l'esecuzione dei due metodi anche se m2 non è sincronizzato

Comunicazione fra threads

- ❑ la sincronizzazione permette di evitare l'esecuzione contemporanea di parti di codice delicate
 - ❑ evitando comportamenti imprevedibili
- ❑ il multithreading può essere sfruttato al meglio solo se i vari threads possono comunicare per cooperare al raggiungimento di un fine comune
 - ❑ esempio classico: la relazione produttore-consumatore
 - ❑ il thread consumatore deve attendere che i dati da utilizzare vengano prodotti
 - ❑ il thread produttore deve essere sicuro che il consumatore sia pronto a ricevere per evitare perdita di dati
- ❑ Java fornisce metodi della classe `Object`
 - ❑ disponibili in istanze di qualunque classe
 - ❑ invocabili solo da metodi sincronizzati

Locking di un Object

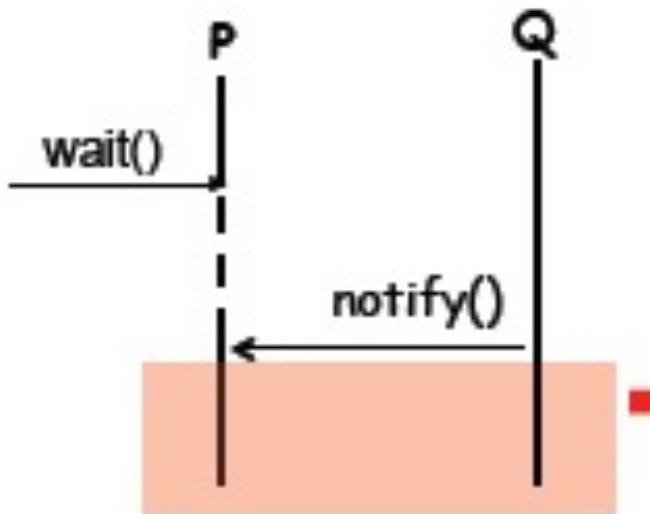
- Il segmento di codice entro un programma che accede ad un oggetto da diversi threads concorrenti sono chiamate “regioni critiche”.
- Regioni critiche
 - Possono essere un blocco o un metodo
 - Sono identificate dalla keyword `synchronized`.
- La piattaforma Java associa un lock ad ogni oggetto che ha del codice `synchronized`.

Wait and Notify Monitor

- È la tipologia di monitor utilizzato dalla JVM:
- Il thread attivo nel monitor può sospendere la sua esecuzione invocando la primitiva `wait()`;
- Quest'ultima ha l'effetto di rilasciare il monitor ed inserire il thread nel "wait set"
- Il thread rimarrà nel "wait set" finchè non verrà invocata una `notify()` da un altro thread che è attivo nel monitor;

Wait and Notify Monitor

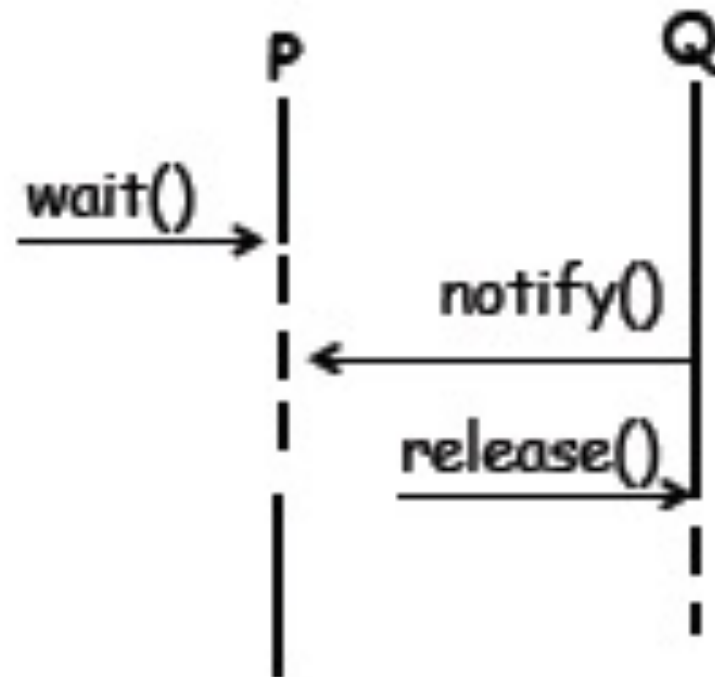
- Quanto un thread esegue la `notify()` può sorgere un problema



Come conseguenza della `notify` entrambi i thread possono, concettualmente, proseguire, violando le proprietà di un monitor !
Uno dei due thread deve essere pertanto sospeso

La soluzione adottata da java

- Tale soluzione prevede che il thread **Q** continui l'esecuzione e che **P** venga **riattivato non appena** Q rilascia il monitor o completa la sua esecuzione.



Signal and Continue

- La soluzione adottata in Java è spesso riferita in letteratura come *"Signal and Continue" monitor* poiché, *differentemente* dalla soluzione di Hoare, il thread che invoca la `notify()` rimane in possesso del monitor e continua la sua esecuzione all'interno della monitor region.

Alcuni problemi

- Il thread che ha eseguito la `notify()` continuando l'esecuzione potrebbe alterare una variabile condivisa con il thread su cui si è invocata la `notify()`;
- Il thread che viene "svegliato" dalla `notify()` potrebbe così operare su uno stato non consistente
- Esempio: un problema di produttori consumatori in cui uno produttore dopo aver invocato una `notify()` altera il contenuto del messaggio depositato

Metodi di Object per la comunicazione fra threads

- ▣ `public final void wait( )`
 - ▣ il thread che invoca questo metodo rilascia il mutex associato all'istanza e viene sospeso fintanto che non viene risvegliato da un altro thread che acquisisce lo stesso mutex e invoca il metodo `notify` o `notifyAll`, oppure viene interrotto con il metodo `interrupt` della classe `Thread`
- ▣ `public final void wait (long millis)`
 - ▣ si comporta analogamente al precedente, ma se dopo un'attesa corrispondente al numero di millisecondi specificato in `millis` non è stato risvegliato, esso si risveglia
- ▣ `public final void wait (long millis, int nanos)`
 - ▣ si comporta analogamente al precedente, ma permette di specificare l'attesa con una risoluzione temporale a livello di nanosecondi

Metodi di Object per la comunicazione fra threads

- ▣ `public final void notify ( )`

- ▣ risveglia il primo thread che ha invocato `wait` sull'istanza
- ▣ poiché il metodo che invoca `notify` deve aver acquisito il mutex, il thread risvegliato deve
 - ▣ attenderne il rilascio
 - ▣ competere per la sua acquisizione come un qualsiasi altro thread

- ▣ `public final void notifyAll ( )`

- ▣ risveglia tutti i threads che hanno invocato `wait` sull'istanza
- ▣ i threads risvegliati competono per l'acquisizione del mutex e se ne esiste uno con priorità più alta, esso viene subito eseguito

- Un Thread può decidere di non proseguire l'esecuzione. Può allora volontariamente chiamare il metodo `wait` all'interno di un metodo sincronizzato, chiamare `wait` in un contesto diverso genera un errore
- Quando un thread esegue `wait` si verificano i seguenti eventi:
 1. il thread rilascia il lock .
 2. il thread viene bloccato.
 3. il thread è posto in una coda di attesa .
 4. gli altri thread possono ora competere per il lock

Il metodo notify()

- Nel momento in cui una condizione che ha provocato una attesa si è modificata si può riattivare un singolo thread in attesa tramite notify
- Quando un thread richiama notify():
 1. viene selezionato un thread arbitrario T fra i thread bloccati
 2. T torna fra i thread in attesa del lock
 3. T diventa Eseguibile

```
public synchronized void insert(T item) {  
    while(count==size) {  
        try {  
            wait();  
        } catch(InterruptedException e) {}  
    }  
    ++count;  
    buffer[in] = item;  
    in = (in + 1) % size;  
    if (count==1) notify();  
}
```

```
public synchronized T remove() {  
    while(count==0) {  
        try {  
            wait();  
        } catch(InterruptedException e) {}  
    }  
    --count;  
    T item = buffer[out];  
    out = (out + 1) % size;  
    if(count==size-1) notify();  
    return item;  
}
```

Class Buffer completa

```
public class BoundedBuffer <T>{  
    private int size;  
    private int count, in, out;  
    private T[] buffer;  
    public BoundedBuffer(int dim) {  
        count = 0;  
        in = 0;  
        out = 0;  
        size = dim  
        buffer = new T [size];  
    }  
    synchronized T remove() {...}  
    synchronized void insert(T item) {...}  
}
```

Class Buffer: main

```
public class ProducerConsumer{  
    public static void main(String args[]){  
        BoundedBuffer<Integer> buffer = new  
        BoundedBuffer<Integer>(10);  
        Producer producer = new Producer(buffer);  
        Consumer consumer = new Consumer(buffer);  
        producer.start();  
        consumer.start();  
    }  
}
```